# Lab#02
# Abstract Data types and Single Linked List

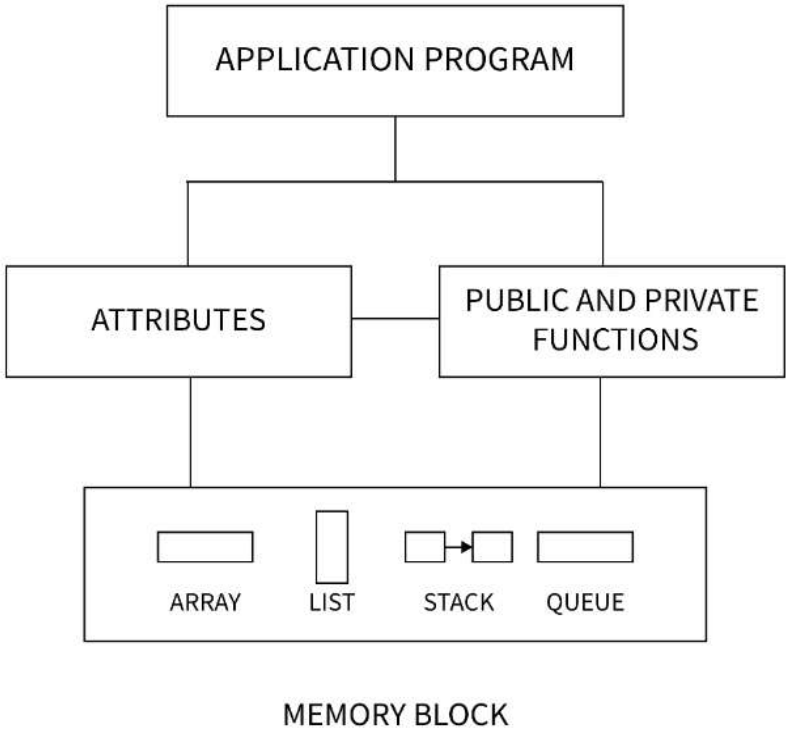
## 2.1 Introduction to ADT

Data types are used to define or classify the type of values a variable can store in it. Moreover, it also describes the possible operations allowed on those values. For example, the integer data type can store an integer value. Possible operations on an integer include addition, subtraction, multiplication, modulo.

**Abstract data type (ADT) is a concept or model of a data type.** An Abstract Data Type in data structure is a kind of a data type whose behavior is defined with the help of some attributes and some functions. Generally, we write these attributes and functions inside a class or a structure so that we can use an object of the class to use that particular abstract data type.

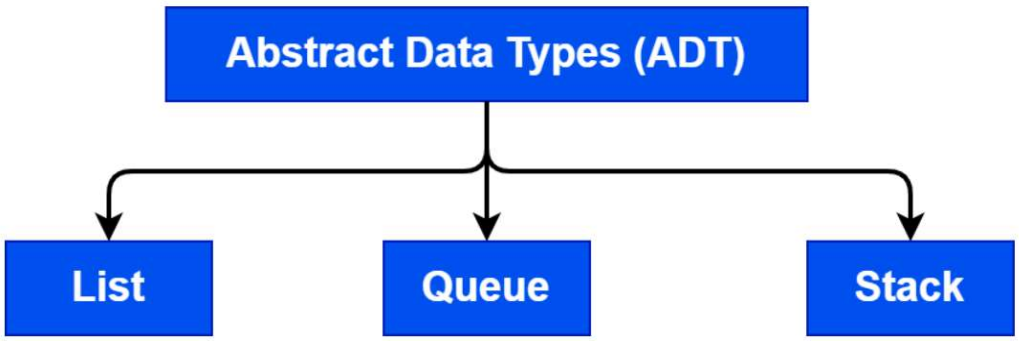
Examples of Abstract Data Type in Data Structure are list, stack, queue etc.

### 2.1.1 Abstract Data Type Model



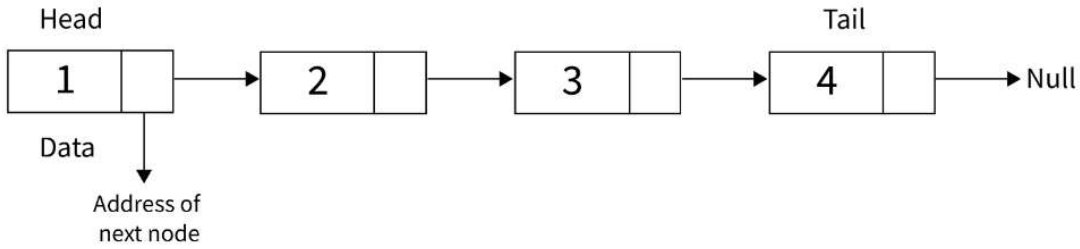

In every programming language, we implement ADTs using different methods and logic. Although, we can still perform all the associated operations which are defined for that ADT irrespective of language. For example, in C, ADTs are implemented mostly using structure. On the other hand, in C++ or JAVA, they're implemented using class. **However, operations are common in all languages.**

**There are mainly three types of ADTs:**




## 2.2 Introduction to List

A Linked List is a linear data structure consisting of connected nodes where each node has corresponding data and a pointer to the address of the next node. The first node of a linked list is called the **Head**, and it acts as an access point. On the other hand, the last node is called the **Tail**, and it marks the end of a linked list by pointing to a *NULL* value!



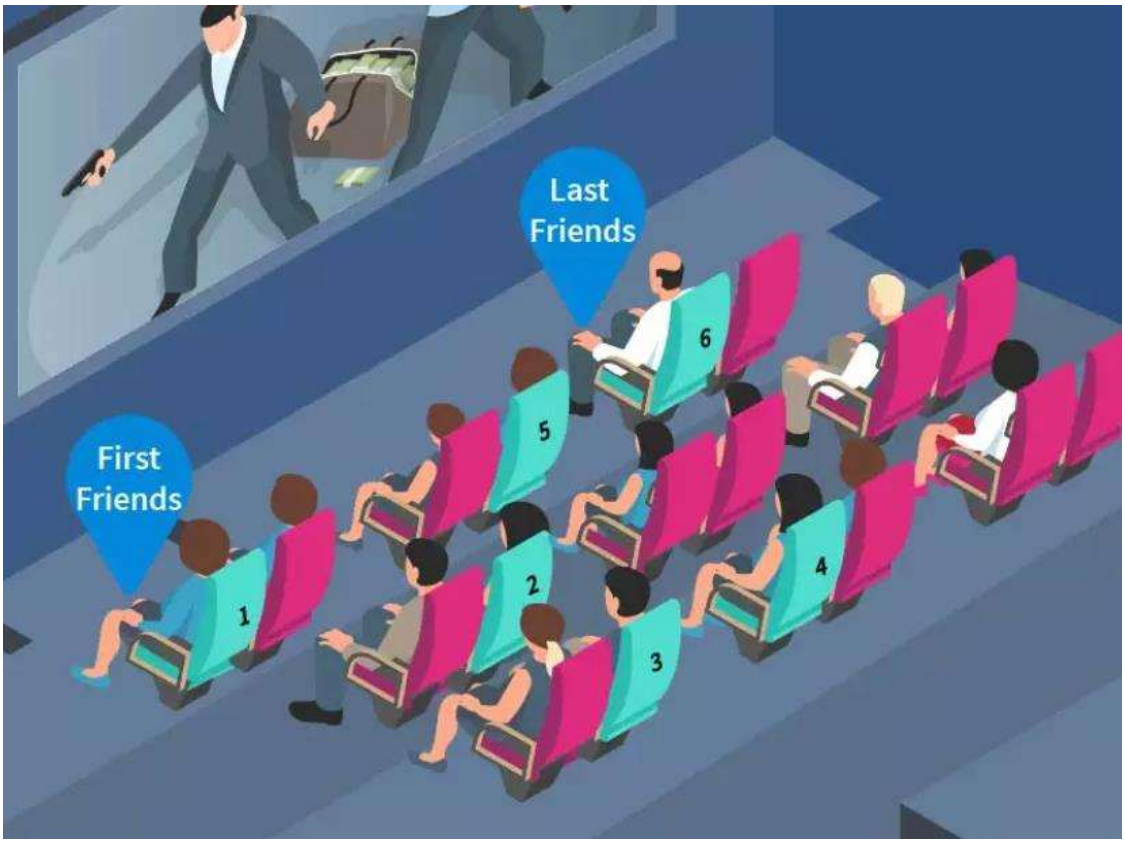

### 2.2.1 Why Do We Need a Linked List?

Imagine going to a movie theater along with a large group of friends, only to find out there's no way to book consecutive seats to accommodate all. However, there are plenty of disjoint empty seats for that particular screen. So, you all buy tickets and sit accordingly.

But there's a catch!

Since only you were buying popcorn for all, how would you distribute that to your friends? You can't remember where everyone sat, and it's dark inside the hall.

So, instead of you remembering where everyone sat, if everyone kept track of the next friend's seat, then such reference would help popcorn tubs to traverse up to a friend who's yet to get popcorn. This could be achieved by passing popcorn to the next friend starting from you!




See how the ***linked list*** came in handy while keeping track of all these randomly distributed disjoint seats?

In a computer, you can think of memory as a movie theatre, the virtual address space as the seats, and any particular memory address as a seat number.

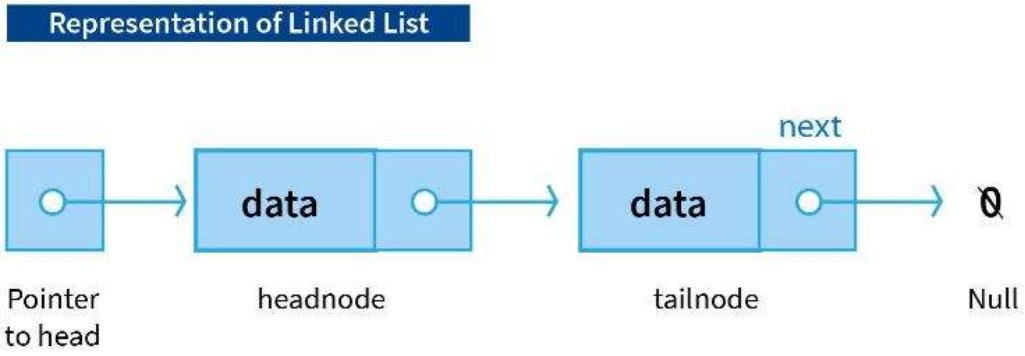
Now, if we had initiated booking in advance, we could have booked an entire row of consecutive seats much like an *array* does when we declare it with a fixed size in advance, like so:

int seats[24];

In our context of a movie theater, if more friends later decide to join the group to catch the movie. We cannot simply append them to the array of seats since there might be other audiences seating just after the last allocated seat.

So either we find another larger empty row of consecutive seats so that all friends can sit together (much like *dynamic array*), or we rely on an approach similar to the **linked list**, in that we simply insert new friends in vacant spots but still have references to them should we require to pass popcorn and drinks.

You see, arrays are suitable when we need fast access. Like who's seating at seat no. 13 can be answered in constant time! But arrays require you to declare a fixed size at the compile-time, due to which memory can be either wasted or fell short.




Whereas linked list shines when we need to modify existing data by insertion and deletion because it doesn't have a fixed size. So, our memory consumption is determined at run time as the linked list shrinks and grows dynamically in constant time.

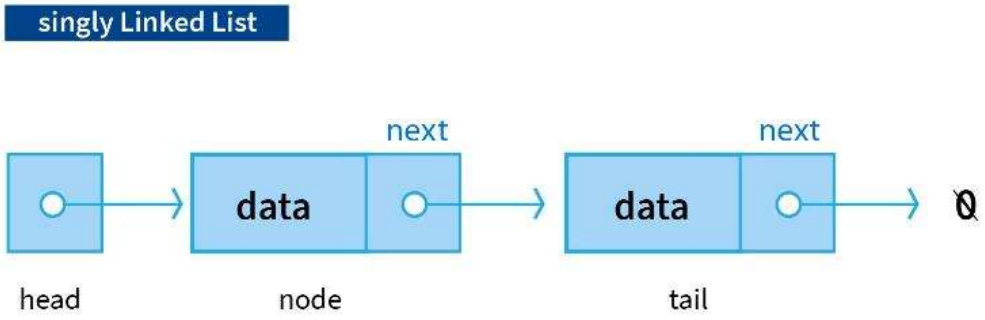
## 2.2.2 Types of Linked List

There are mainly three types of linked lists:

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

### *2.2.2.1 Singly Linked List*

Here, we can only traverse in *one direction* (not the band ) due to the linking of every node to its next node.




### *2.2.2.2How Can We Declare a Linked List?*

*A node which is both head and tail and in and of itself a linked list!*

```
class Node {
        some_constructor(int value) {
                int data = value;
                Node* next = NULL;
        }
```

```
}
```

where data can be any *valid type* and not just *integer*, but next must be a *Node pointer*.

Let's declare the **head** of our linked list like so:

```
Node* head = new Node(77);
```

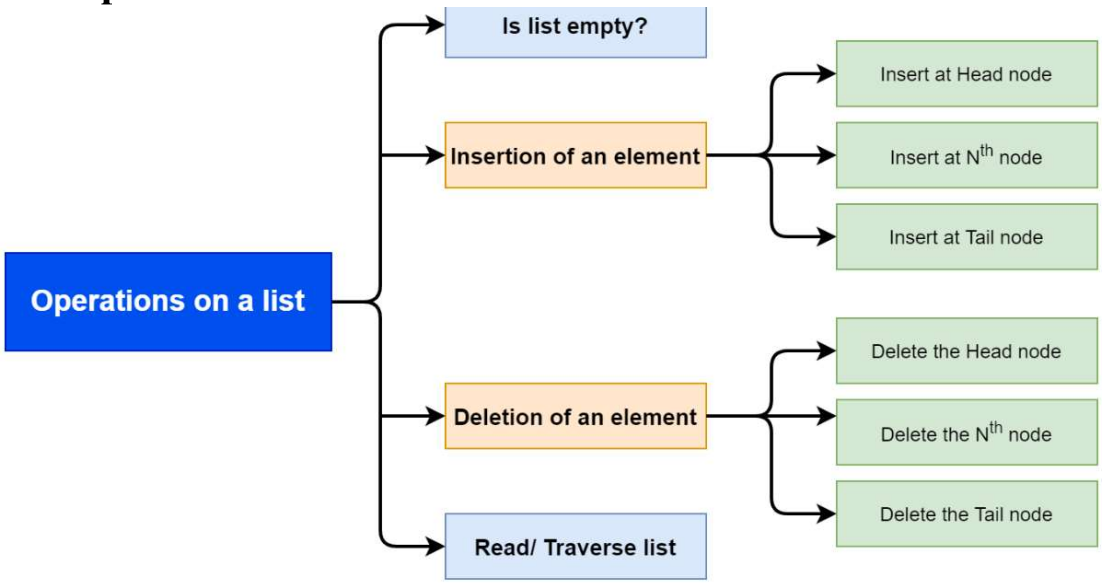
And that's it for a single node of a singly linked list

*"But how can we extend it for a linked list of size k?"*

```
Node* head = new Node(num[0]);
Node* curr = head;
for itr -> 1 to (k - 1) {
        curr.next = new Node(num[itr]);
        curr = curr.next;
```

where num could be an array of integers storing the data for building our respective linked list and itr could be any variable iterating over the indices of the said array.

## 2.2.3 Operations on Linked List



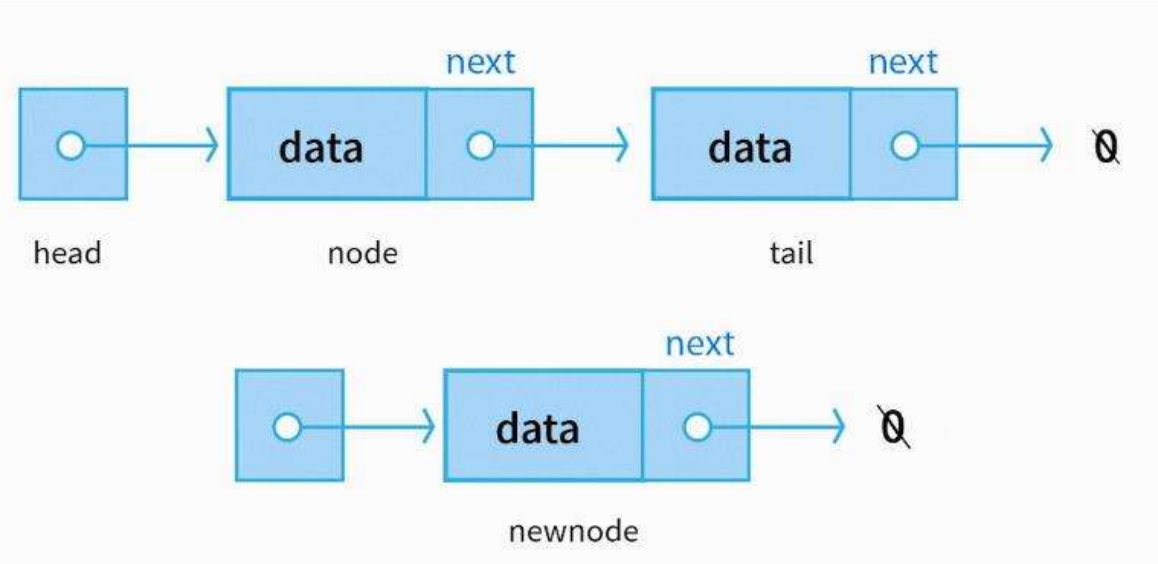

Some of the most essential operations defined in List ADT are listed below.

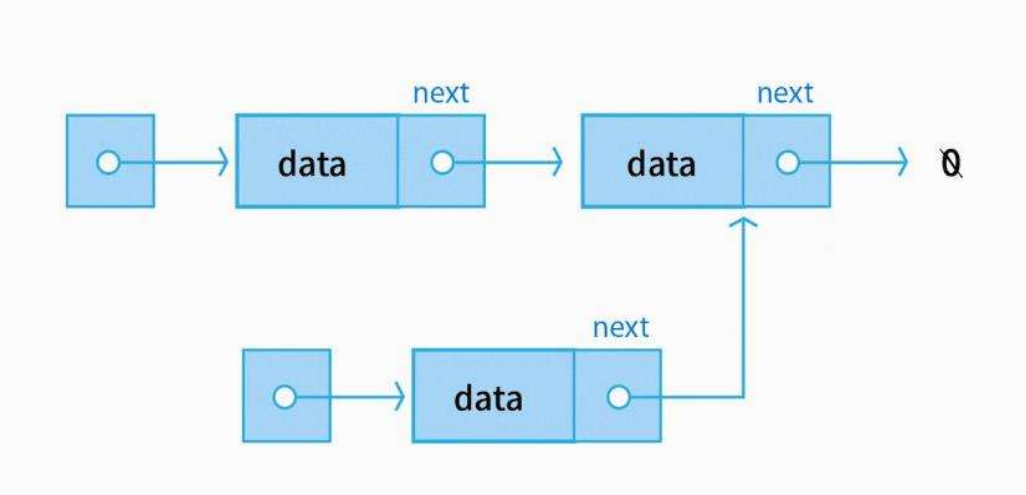
- **front():** returns the value of the node present at the front of the list.
- **back():** returns the value of the node present at the back of the list.
- **push_front(int val):** creates a pointer with value = val and keeps this pointer to the front of the linked list.
- **push_back(int val):** creates a pointer with value = val and keeps this pointer to the back of the linked list.
- **pop_front():** removes the front node from the list.
- **pop_back():** removes the last node from the list.
- **empty():** returns true if the list is empty, otherwise returns false.
- **size():** returns the number of nodes that are present in the list.

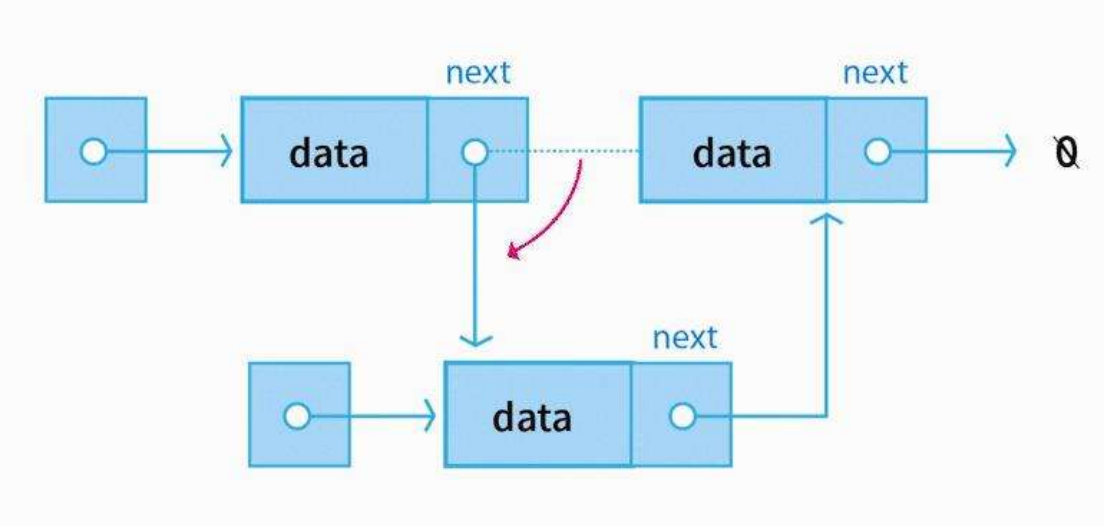
#### *2.2.3.1 Insertion*

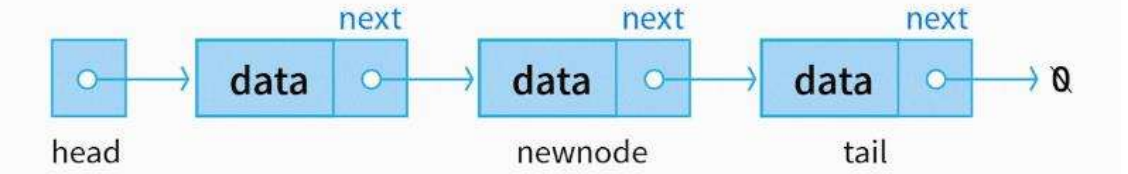
Inserting a new node at a given position is achieved by manipulating at most 2 next pointers like so; consider two pointers, prevNode, and newNode set newNode's next pointer to the prevNode's next pointer and set prevNode's next pointer to the newNode where prevNode denotes a pointer to the node after which newNode is to be inserted.
And since only a constant number of pointers gets changed irrespective of the size of the linked list, the complexity should have been O(1).

However, to get access at any given position, we have to traverse starting from the head till prevNode. Thus, **the worst case** for insertion could be expressed as; Traversal O(n) + Just the insertion O(1) = Insertion O(n). However, **the best case** would be to insert in the beginning and hence the time complexity becomes Ω(1).
Notice that the order of execution does matter in case of insertion. If we interchange the steps i.e. first set prevNode's next pointer to the newNode then we lose the reference to the remaining of the linked list previously occurring after the prevNode. Something to keep in mind!


next
next
data
data
head
node
tail
next
data
newnode


next
next
data
data
next
data

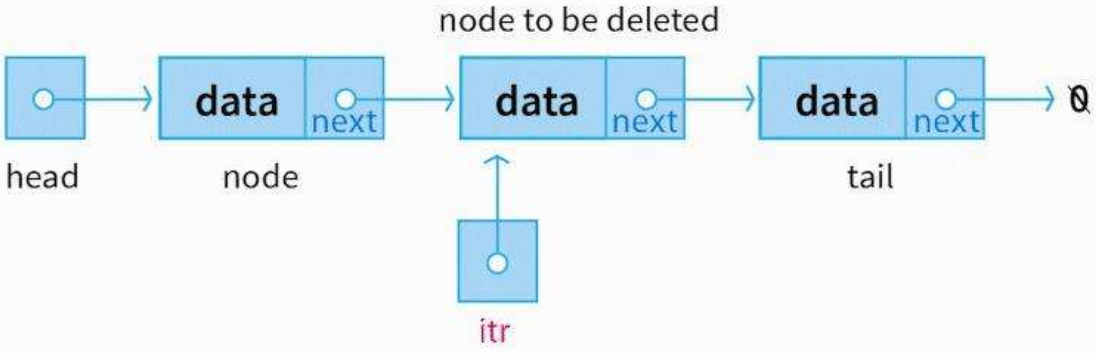

next
next
data
data
next
data

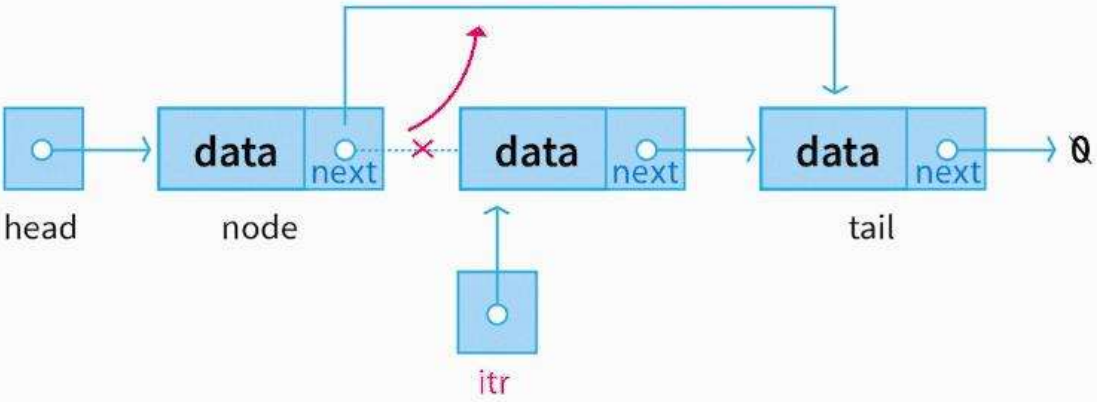


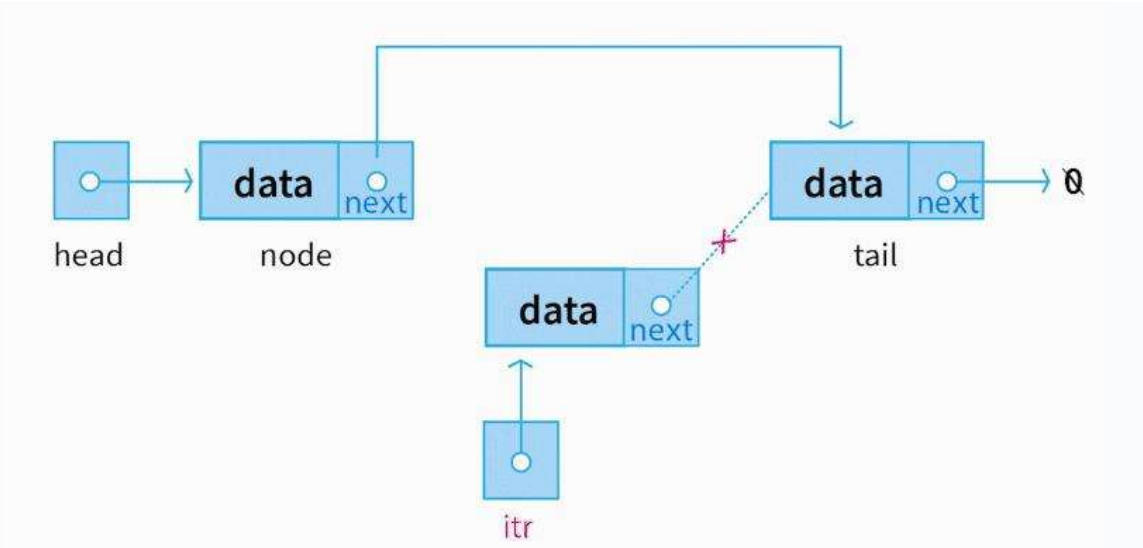

### *2.2.3.2 Deletion*

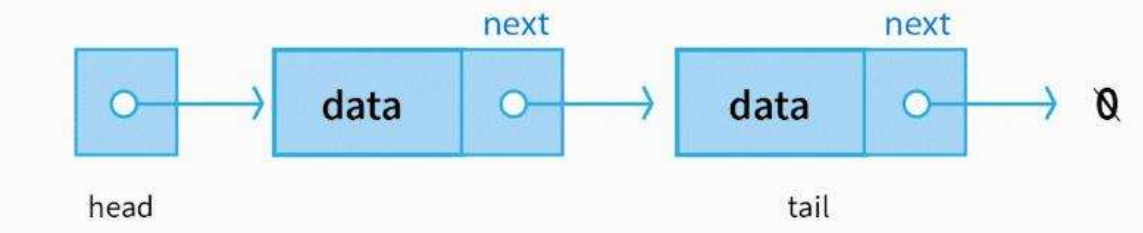
We can delete a node located at a specified index by manipulating at most 2 next pointers like so; consider two pointers, prevNode, and targetNode set prevNode's next pointer to the targetNode's next pointer and set targetNode's next pointer to NULL where prevNode denotes a pointer to the node after which targetNode is to be deleted. Following the same reasoning as insertion, the best case and the worst case time complexity stands at Ω(1) and O(n) respectively.

Again just like insertion, the order is important. It's important to realize that setting targetNode's next pointer to NULL as the first step would cost us the reference to the remaining of the linked list (if the targetNode wasn't the *Tail*).



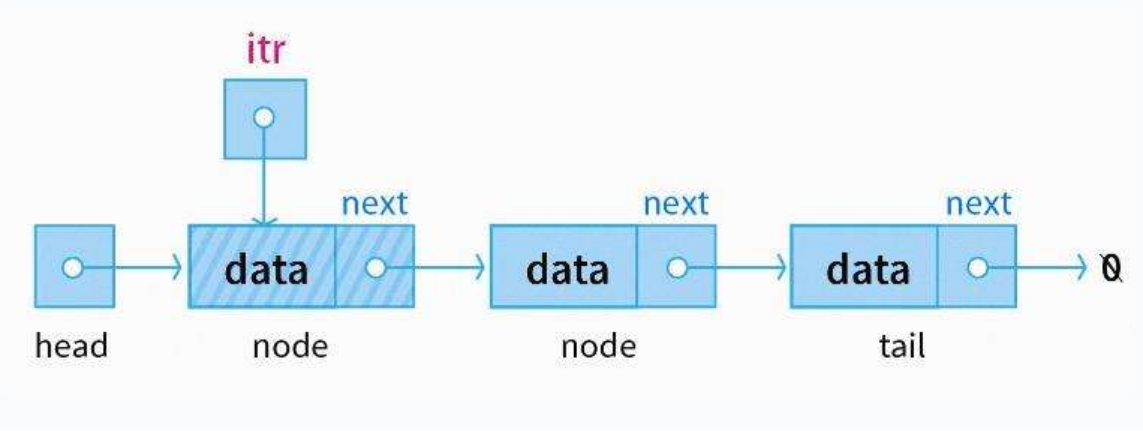



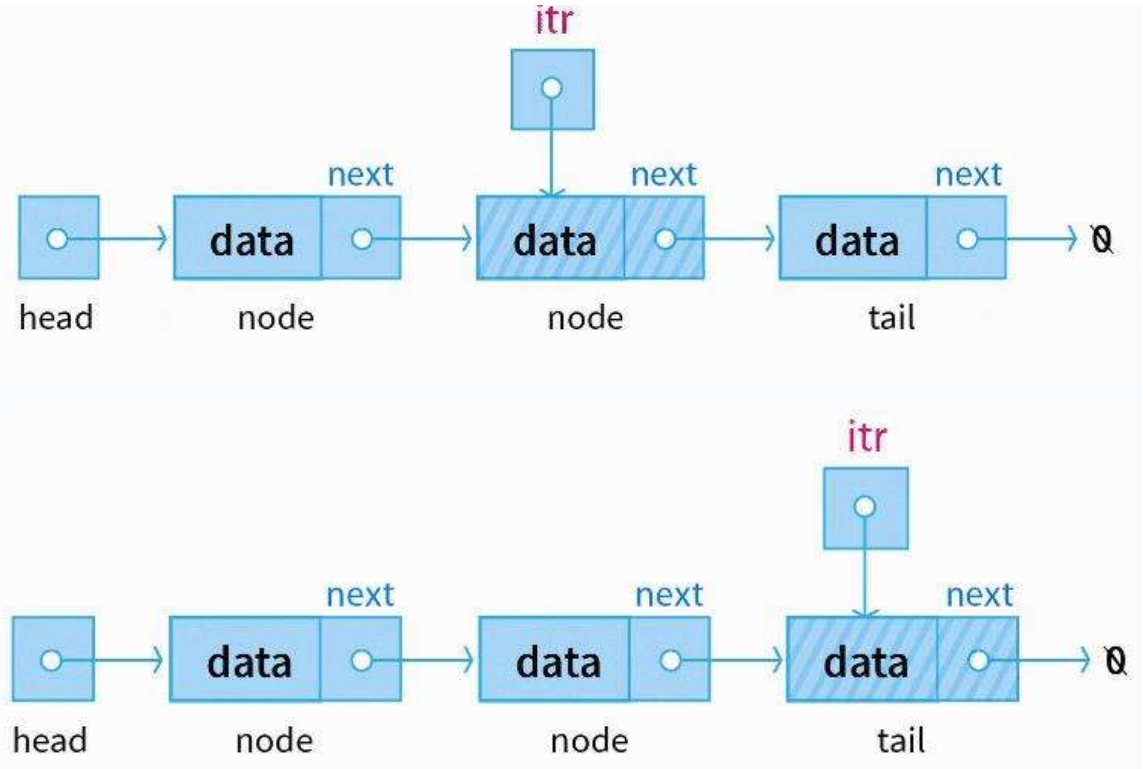



### *2.2.3.3 Traversal*

We can traverse the entire linked list starting from the head node. If there are n nodes then the time complexity for traversal becomes O(n) as we hop through each and every node.






***2.2.3.4 Search***

Given a particular **key** (data), we need to search for a node whose data matches the key. In the best-case scenario, the head would be the node we are looking for, whereas in the worst-case the required node would be the tail. In terms of time complexity the best case, the average case, and the worst case of searching are denoted as Ω(1), Θ(n), and O(n) respectively.

## 2.2.4 Implementation of Linked List in C++

```
struct Node {
    // Blueprint for a node of a linked list
    int data;
    Node *next;
    Node() : data(0), next(nullptr) {}
    Node(int x) : data(x), next(nullptr) {}
    Node(int x, Node *next) : data(x), next(next) {}
};

class LinkedList {
public:
    // constructor for creating a linked list
    // and storing it in 'head'
    LinkedList() {}

    void traversal() {
```

```
        Node* itr = head;
        while (itr != nullptr) {
            cout << itr -> data << endl;
            itr = itr -> next;
        }
    }

    // 'newNode' - pointer to the node to be inserted
    // 'prevNode' - pointer to the node after which insertion occurs
    void insertion(Node* prevNode, Node* newNode) {
        newNode -> next = prevNode -> next;
        prevNode -> next = newNode;
    }

    // 'targetNode' - pointer to the node to be deleted
    // 'prevNode' - pointer to the node after which deletion occurs
    void deletion(Node* prevNode, Node* targetNode) {
        prevNode -> next = targetNode -> next;
        targetNode -> next = null;
    }

    bool search(int key) {
        Node* itr = head;
        while (itr != null) {
            if (itr -> data == key)
                return true;
            itr = itr -> next;
        }
        return false; // key not found!
    }
}
```

## 2.2.5 Time Complexity of Linked List

As you can clearly see that few linked list operations are more efficient than others. To understand, let's observe the dichotomy of linked lists and arrays in the context of time complexity for the aforementioned operations.

| Operation | linked list | Array |
|---|---|---|
| Random access | O (n) | O (1) |
| Insertion and Deletion at the beginning | O (1) | O (n) |
| Insertion and Deletion at the end | O (n) | O (1) |
| Insertion and Deletion from random location | O (n) | O (n) |

Time Complexity (Big O) Comparisons

## 2.2.6 Advantages / Disadvantages of Linked List

**Dynamic in nature** - Linked lists are dynamic in nature, and their size can be adjusted according to our requirements.

**Ease of Insertion and Deletion** - Insertion and deletion of a node in a linked list remain efficient as the nodes are stored in random locations, and we only need to update the next pointer of a constant number of nodes.

**Memory efficient** - Memory consumption of a linked list is efficient as its size can grow or shrink dynamically according to our requirements.

**Implementation** - Various advanced data structures can be implemented using a linked list vis-a-vis *stack*, *queue*, *graph*, *hash maps*, etc.

### Disadvantages of Linked List

**Memory usage** - A node in a linked list occupies more memory than an element in an array as each node occupies at least two kinds of variables.

**Accessing a node** - If you want to access a node in a linked list, you have to traverse starting from the head. We cannot access any random nodes directly except for the head itself, since nodes don't share a linear order in the physical memory and are only referenced by pointers.

**Traversing in reverse order** - Traversing in reverse order is difficult. Although a doubly-linked list makes it easier but requires more memory to store those extra 'prev' pointers.

## 2.2.7 Applications of Linked List

It would have been futile if a linked list didn't have many applications beyond a mere movie theater use-case.

Here are some of the applications of a linked list:

- Linear data structures such as stack, queue, and non-linear data structures such as hash maps, graphs can be implemented using linked lists.
- Doubly linked lists are useful for building a forward-backward navigation button in a web browser or undo-redo feature in an editor.
- Circular Doubly linked lists can also be used for the implementation of advanced data structures like Fibonacci Heap.

---

# Practice Problems:

**Lab Activity 1:**

Write a C++ function to count the number of nodes in a singly linked list.

You can take the following structure as a reference:

```
struct Node {
    int data;
    Node* next;
};
```

Your task is to implement the function with the following prototype:
**int countNodes(Node* head);**
The function should take the head of a singly linked list as an argument and return the total number of nodes present in the list.

**Lab Activity 2:**

Write a C++ function to check if a given singly linked list is empty. You can use question#01 structure as reference.

Your task is to implement the function with the following prototype:

**bool isListEmpty(Node* head);**

The function should take the head of a singly linked list as an argument and return `true` if the list is empty (i.e., it has no nodes), and `false` otherwise.

**Lab Activity 3:**

Write a C++ function to delete a node with a given value from a singly linked list. Assume that the value to be deleted is always present in the list.